

PART A: VARIOUS ATTENTION MODELS

1. What is Attention?

Attention allows a model to focus on important parts of the input instead of treating everything equally.

General idea:

$$\text{Attention}(Q, K, V) = \sum \text{weight} \times \text{Value}$$
$$\text{Attention}(Q, K, V) = \sum \text{weight} \times \text{Value}$$

Where:

- Q = Query
- K = Key
- V = Value

2. Additive Attention (Bahdanau Attention)

✦ Used in: Early seq2seq RNN models (machine translation)

Mechanism

- Uses a feed-forward neural network to compute attention scores.

Equation

$$e_{ij} = v^T \tanh(W_q q_i + W_k k_j)$$
$$e_{ij} = v^T \tanh(W_q q_i + W_k k_j)$$
$$\alpha_{ij} = \text{softmax}(e_{ij})$$
$$\alpha_{ij} = \text{softmax}(e_{ij})$$

Figure

Query → FFN → Score

Key → FFN →

Pros / Cons

✓ Works well for small sequences

✗ Computationally expensive

3. Dot-Product Attention

Mechanism

- Attention score is computed using dot product.

Equation

$$\text{score}(Q, K) = Q \cdot K$$

$$\text{Kscore}(Q, K) = Q \cdot K$$

Figure

$Q \cdot K \rightarrow \text{Softmax} \rightarrow \text{Attention Weights}$

✓ Faster than additive attention

✗ Large values for high dimensions

4. Scaled Dot-Product Attention (Transformer Attention)

✦ Used in: Transformers, ViT

Equation

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Why scaling?

- Prevents extremely large dot products
- Stabilizes gradients

Figure

$$Q \text{ — } K^T / \sqrt{d} \rightarrow \text{Softmax} \rightarrow \times V$$

5. Self-Attention

✦ Key idea:

Query, Key, and Value come from the same input.

Use

- Captures long-range dependencies
- Used in Transformers and ViT

Figure

Input Tokens

$$Q = K = V$$

Self-Attention

6. Multi-Head Attention (MHA)

✦ Key idea:

Use multiple attention heads to focus on different relationships.

Equation

$$\text{MHA}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_O$$
$$\text{MHA}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

Figure

Head1 → Attention

Head2 → Attention

Head3 → Attention

↓

Concatenate → Linear

✓ Captures diverse features

✓ Core of Transformers and ViT

7. Cross-Attention

✦ Used in: Encoder-decoder models

Mechanism

- Queries from decoder
- Keys & values from encoder

Figure

Decoder Query → Attention → Encoder Output

8. Local / Window Attention

✦ Used in: Swin Transformer

Idea

- Attention computed within local windows
- Reduces complexity

Figure

[Window 1] [Window 2]

[Window 3] [Window 4]

✓ Efficient

✓ Preserves locality

9. Global Attention

- Attention over all tokens
- High computational cost
- Used in vanilla ViT

PART B: ViT (Vision Transformer) HYPERPARAMETERS

ViT performance depends heavily on design hyperparameters.

1. Image Size ($H \times W$)

- Common: 224×224
- Larger images $\rightarrow$ more tokens $\rightarrow$ higher cost

2. Patch Size ($P \times P$)

✦ Image is divided into fixed-size patches.

Number of patches = $\frac{H \times W}{P^2}$ Number of patches = $P^2 \times \frac{H \times W}{P^2}$

Example

- Image: 224×224
- Patch: 16×16
- Tokens = $\frac{224 \times 224}{16^2} = 196$

Trade-off

Small Patch Large Patch

More detail Less computation

More tokens Less memory

3. Patch Embedding Dimension (D)

- Dimension of each patch vector
- Typical values: 256, 512, 768

✦ Larger D $\rightarrow$ more representation power but slower training

4. Number of Transformer Layers (Depth)

- Number of encoder blocks
- Examples:
 - ViT-Base: 12

- ViT-Large: 24

✓ More layers $\rightarrow$ better modeling

✗ Risk of overfitting

5. Number of Attention Heads (H)

- Used in Multi-Head Attention
- Typical values: 8, 12, 16

Constraint:

$$d_{\text{head}} = \frac{D}{H} \quad d_{\text{head}} = H \times D$$

6. MLP Hidden Dimension

- Used in Feed-Forward Network inside transformer
- Usually:

$$\text{MLP dim} = 4 \times D$$

$$\text{DMLP dim} = 4 \times D$$

Example:

- $D = 768 \rightarrow \text{MLP} = 3072$

7. Positional Embedding

- Adds spatial information
- Types:
 - Learnable
 - Sinusoidal
 - Relative (Swin)

✦ Necessary because transformers are permutation-invariant

8. Dropout Rate

- Prevents overfitting
- Common values: 0.1 – 0.3

Used in:

- Attention
- MLP layers

9. Classification Token ([CLS])

- Extra token appended to patch tokens
- Final representation used for classification

10. Attention Type (ViT-specific)

Attention Type Used in

Global ViT

Window Swin

Shifted Window Swin

Hybrid CvT

Exam-Ready Summary
Attention Models

- Additive attention (Bahdanau)
- Dot-product attention
- Scaled dot-product attention
- Self-attention
- Multi-head attention
- Cross-attention
- Local & global attention

ViT Hyperparameters

- Image size
- Patch size
- Number of patches
- Embedding dimension
- Transformer depth
- Number of heads
- MLP dimension
- Positional embeddings
- Dropout
- CLS token

Attention Model Hyperparameters

Attention hyperparameters control how attention is computed and learned in transformer models.

Main Attention Hyperparameters:

1. Embedding Dimension (d_{model})
 - Size of input feature vector for each token
 - Example: 256, 512, 768
2. Number of Attention Heads (h)
 - Number of parallel attention mechanisms
 - More heads → learn multiple relationships
 - Typical values: 8, 12, 16
3. Key / Query / Value Dimensions (d_k, d_q, d_v)
 - Dimensionality of Q, K, V vectors
 - Usually:

$$d_k = d_q = d_v = d_{\text{model}} \quad h_{d_k} = d_q = d_v = h_{d_{\text{model}}}$$

$$\frac{d_k}{h_{d_k}} = \frac{d_q}{h_{d_q}} = \frac{d_v}{h_{d_v}} = \frac{d_{\text{model}}}{h_{d_{\text{model}}}}$$

4. Number of Transformer Layers (L)
 - Number of stacked attention + FFN blocks
 - More layers → better representation
5. Feed-Forward Network Size (d_{ff})
 - Hidden layer size in FFN
 - Typically 2–4× of embedding dimension
6. Dropout Rate
 - Prevents overfitting
 - Applied after attention and FFN
7. Attention Mask

- Prevents attention to padding or future tokens

Vision Transformer (ViT) Hyperparameters

ViT hyperparameters control how images are converted into patches and processed.

Main ViT Hyperparameters:

1. Image Size ($H \times W$)
 - Input image resolution
 - Example: 224×224
2. Patch Size ($P \times P$)
 - Image divided into fixed-size patches
 - Example: 16×16
3. Number of Patches (N)

$$N = \frac{HW}{P^2} \quad N = \frac{P^2}{HW}$$
4. Patch Embedding Dimension (D)
 - Dimension of each patch vector
 - Example: 768
5. Number of Transformer Encoder Layers
 - Controls depth of ViT
6. Number of Attention Heads
7. MLP Hidden Dimension
 - Hidden layer size of feed-forward network
8. Positional Embedding Type
 - Fixed or learned positional encoding
9. CLS Token
 - Special token used for image classification

Describe Various Attention Models (5 marks)

Attention mechanism allows a model to focus on the most important parts of the input while making a prediction. Different attention models have been proposed depending on task and architecture.

Additive Attention (Bahdanau Attention)

- Introduced for encoder–decoder RNN models
- Uses a feed-forward neural network to compute attention scores
- Works well for small dimensions

Score function:

$$\text{score}(h_t, h_s) = v^T \tanh(W_1 h_t + W_2 h_s) \quad \text{score}(h_t, h_s) = v^T \tanh(W_1 h_t + W_2 h_s)$$

Features:

- More flexible
- Computationally expensive

Example: Machine translation with RNNs

2 Multiplicative (Dot-Product) Attention (Luong Attention)

- Uses **dot product** between query and key
- Faster than additive attention

Score function:

$$\text{score}(Q,K)=QTK\text{score}(Q,K)=Q^T K\text{score}(Q,K)=QTK$$

Features:

- Simple and efficient
- Less effective for large dimensions

3 Scaled Dot-Product Attention

- Used in **Transformers**
- Prevents large dot-product values

Equation:

$$\text{Attention}(Q,K,V)=\text{Softmax}(QK^Tdk)V\text{Attention}(Q,K,V)=\text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V\text{Attention}(Q,K,V)=\text{Softmax}(dkQK^T)V$$

Features:

- Stable gradients
- Efficient parallel computation

4 Self-Attention

- Query, Key, and Value come from the **same input**
- Captures relationships within a sequence

Example:

- Understanding word dependencies in a sentence
- Vision Transformer (ViT)

Advantage:

- Handles long-range dependencies efficiently

5 Multi-Head Attention

- Uses **multiple attention heads in parallel**
- Each head learns different relationships

Equation:

$$\text{MultiHead}(Q,K,V)=\text{Concat}(\text{head}_1,\dots,\text{head}_h)W^OMultiHead(Q,K,V)=\text{Concat}(\text{head}_1,\dots,\text{head}_h)W^O$$

Benefits:

- Rich representation
- Captures multiple contexts simultaneously

6 Global Attention

- Attends to **all encoder states**
- High accuracy but high computation

Used in: Seq2Seq models

7 Local Attention

- Attends to a **small window** of encoder states
- Faster than global attention

Used when: Long sequences exist